

COLISTREAM: A Streaming Execution-Cache Architecture for Mixture-of-Experts Inference on Consumer GPUs

Brian Dean Ullery

July 14, 2026

Abstract

Mixture-of-Experts (MoE) language models decouple parameter count from per-token compute, but their working set still dwarfs consumer GPU memory: a 744B-parameter model activates only $\sim 40\text{B}$ parameters per token, yet its routed experts occupy $\sim 370\text{ GB}$ at 4-bit precision. Existing offloading systems treat VRAM as *storage* and ask which weights fit; COLISTREAM instead treats the GPU as a *streaming compute device* and VRAM as an *execution cache* at the top of a managed SSD $\rightarrow$ RAM $\rightarrow$ VRAM $\rightarrow$ SM hierarchy. Expert weights flow through the hierarchy as tile-chunked asynchronous DMA; compute is event-driven at sub-expert granularity, so matrix multiplication begins while later tiles of the same expert are still on the bus; a two-lane PCIe scheduler gives speculative prefetch strictly tile-bounded interference with demand traffic; and a recency-primary, decayed-frequency replacement policy retains temporally local experts in VRAM. We give a pipeline model with explicit fill/drain bounds, a closed-form tile-size feasibility window that we fit to measurements ($R^2=0.99$), a proof that frequency-primary replacement suffers unbounded pollution under working-set turnover (observed as a $7.6\times$ slowdown before the fix), and a Che-approximation analysis of expected cache behavior under Zipfian expert routing. Implemented inside the open-source colibrì engine and validated token-exactly against a reference implementation, COLISTREAM sustains 26.6–27.1 GB/s of cold expert streaming with compute fully hidden ($\sim 2,250$ experts/s) and $\sim 16,000$ experts/s from the warm execution cache on a single RTX 5070 Ti, and is $1.065\times$ faster end-to-end than the optimized CPU expert path on a deliberately adversarial, disk-bound workload whose theoretical compute-side ceiling is $1.05\times$.

1 Introduction

Sparse Mixture-of-Experts architectures [18, 12, 8] scale language models to trillions of parameters while activating only a small routed subset per token, a design carried to frontier scale by DeepSeek-V3 [5] and GLM-5.2 [21]. This sparsity creates a peculiar systems opportunity: per-token *compute* fits comfortably on a consumer GPU, while per-token *weights* do not. GLM-5.2 routes $k=8$ of 256 experts in each of $L=75$ MoE layers; at 4-bit precision each expert is $s \approx 18.9\text{ MB}$, so a cold token demands $L k s \approx 11.4\text{ GB}$ of expert weights — against 16 GB of VRAM and 370 GB of model.

The dominant response is *placement*: decide which tensors reside in VRAM, which in RAM, and which on disk, then compute each tensor where it lives [19, 2, 17]. The colibrì engine [10], on which this work builds, exemplifies the placement view on CPUs: a RAM LRU cache and a learned pinned set front a disk-resident expert store, and its documentation explicitly rejects per-use GPU copies because a synchronous NVMe $\rightarrow$ VRAM copy “would only replace the disk bottleneck with a PCIe bottleneck.”

COLISTREAM is the answer to that objection. Its premise is that the copy is only a bottleneck if it is *synchronous*. If expert weights are treated as a *stream* — tile-chunked, double-buffered, event-ordered — then storage, RAM, the PCIe link, and the GPU operate as concurrent pipeline stages, and per-token latency collapses from the *sum* of stage times toward their *maximum* (Theorem 4.3). VRAM stops being storage and becomes what a register file is to a cache hierarchy: the place where bytes happen to be at the moment they are consumed.

Concretely, this paper makes the following contributions:

1. **Architecture** (§3). A VRAM *execution cache*: a pool of fixed-size expert slots fed by tile-chunked asynchronous DMA on dedicated copy lanes, with per-slot CUDA events splitting each expert so gate/up projections compute while the down projection is still in flight; a two-phase demand submit that overlaps PCIe transfers with disk reads; a tile-granular soft-preemption scheduler for speculative traffic; and four prefetch sources (current router, cross-layer router lookahead, temporal locality, long-term heat) mapped onto two tiers.
2. **Analysis** (§4). A deterministic pipeline bound with explicit fill/drain terms; a closed-form tile-size feasibility window balancing per-copy overhead against preemption latency, fitted to hardware measurements; a proof that frequency-primary replacement without decay suffers total pollution under working-set turnover, motivating COLISTREAM’s recency-primary decayed policy; and an expected-hit-rate analysis via the Che approximation [4] under Zipfian expert popularity.
3. **Implementation and evaluation** (§5, §6). A complete implementation inside colibrì [10] (CUDA backend + engine integration, ~ 900 lines), validated *token-exactly* against a reference transformer implementation, with a fallback lattice in which every failure degrades to the CPU path rather than to a wrong answer. On an RTX 5070 Ti we measure 26.6–27.1 GB/s sustained cold streaming with compute hidden, $\sim 16,000$ experts/s warm, a $7.6\times$ replacement-policy ablation, and a carefully controlled end-to-end comparison on the real 744B model, including an honest account of two confounds (speculative-decoding defaults and greedy-path divergence) that initially inflated the measured advantage from $1.065\times$ to $1.48\times$.

2 Related Work

MoE models. Sparsely gated expert layers were introduced at scale by Shazeer et al. [18] and industrialized by GShard [12] and Switch Transformers [8]. DeepSeek-V3 [5] established the modern frontier recipe — fine-grained experts, sigmoid routing with auxiliary-loss-free balancing, multi-token prediction — that GLM-5.2 [21] follows.

Offloading systems. FlexGen [19] formulates dense-LLM offloading as a scheduling problem over a GPU/CPU/disk hierarchy, optimizing throughput for large batches; DeepSpeed-Inference [2] and ZeRO-Offload [17] stream dense weights past the GPU. COLISTREAM differs in exploiting MoE *selectivity*: only the routed k/E fraction of each layer crosses the bus, which changes the economics from “stream everything every token” to “cache what routing repeats.”

MoE offloading. Eliseev and Mazur [7] cache experts in VRAM with LRU and speculate on next-layer routing for Mixtral-class models; MoE-Infinity [20] traces activation patterns to prefetch experts; Fiddler [11] executes cold experts on the CPU instead of moving them. COLISTREAM synthesizes all three ideas and adds what they lack: sub-expert tile streaming with event-driven

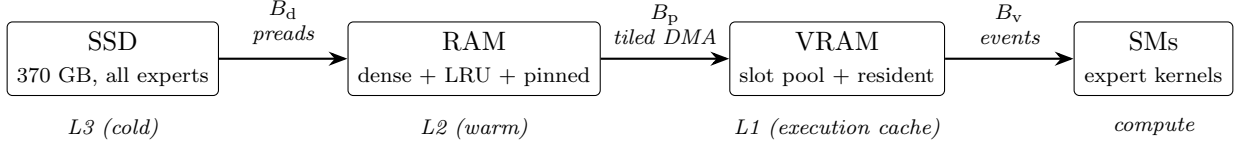


Figure 1: The COLISTREAM hierarchy. Every level behaves as a cache over the level below; no expert permanently occupies VRAM except a measured hot set.

partial compute, an explicit two-lane PCIe scheduler with bounded speculative interference, a replacement policy derived from a pollution analysis, and a disk tier below RAM (370 GB does not fit in RAM, unlike Mixtral).

Caching theory. Replacement is classical: Belady’s optimal policy [3], LRU-K [16], and ARC [14] inform §3.7; the Che approximation [4] underlies our hit-rate model. Speculative decoding [13] and weight quantization [9, 6] are orthogonal levers the implementation composes with.

3 The ColiStream Architecture

3.1 Design premise

Traditional GPU inference asks: *which weights fit in VRAM?* COLISTREAM asks: *is every hardware engine — NVMe queue, PCIe copy engine, tensor pipeline — busy at every instant?* VRAM is a high-speed execution cache, not permanent storage (Fig. 1). The scheduler’s objective is that the compute units never stall on memory unless prediction fails *and* every faster tier misses.

3.2 Expert representation and tile streaming

Each routed expert comprises three quantized matrices (gate, up, down projections) plus per-row scales, stored contiguously. Rather than treating the expert as the unit of transfer, COLISTREAM streams it as a sequence of fixed-size *tiles* ($t = 512$ KB by default, configurable 64 KB–4 MB via `--stream-tile-kb`). Tiles serve two purposes:

1. **Partial-compute overlap.** Two CUDA events split each expert: one after the gate/up matrices (two-thirds of the bytes, transferred first), one after the down matrix. The gate/up kernels and the SiLU are enqueued against the first event, the down kernel against the second — so computation of an expert begins while its own tail is still on the bus.
2. **Soft preemption.** A tile is the scheduling quantum of the PCIe link (§3.5): the worst-case delay a demand transfer suffers behind speculative traffic is one tile, not one expert.

3.3 The VRAM execution cache

VRAM is carved into C fixed-size slots ($C=727$ at 18.9 MB on a 16 GB card after the dense reserve), each holding one expert keyed by (layer, expert). Fixed slots eliminate fragmentation and make transfer cost uniform, which in turn drops the transfer-cost term out of replacement comparisons. Slots carry busy bits (in-flight demand groups, in-flight uploads) so eviction can never race an active consumer, and per-slot events make reuse ordering explicit. Container quantization formats

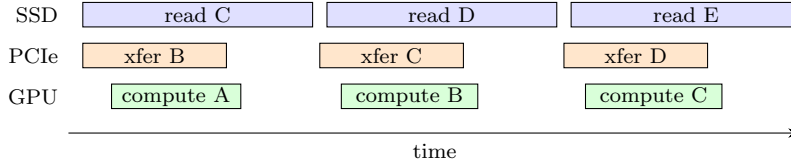


Figure 2: Steady-state timeline. Disk reads expert $n+2$, PCIe transfers $n+1$, and the GPU computes n , concurrently; each engine’s idle time is bounded by the pipeline imbalance, not by the sum of the other stages.

(int8; int4 offset-nibble; int2; f32) are decoded *inside* the kernels, so any mixture streams as-is with no conversion pass.

3.4 Execution pipeline and double buffering

A MoE layer’s demand set (the k routed experts per position, deduplicated batch-wide) is submitted split-phase. Phase 0 submits RAM-resident experts immediately: their DMA and kernels run *while* the I/O pool is still reading the disk misses. Phase 1 submits the misses as their preads land. Within a phase, cache *hits* are enqueued before *misses*, so a resident expert never queues behind another expert’s transfer. All kernels issue to one execution stream; all demand DMA to a dedicated copy stream — while the GPU computes expert j , the copy engine fills expert $j+1$ ’s slot. Classical A/B double buffering is the $C=2$ special case; the slot pool generalizes the same overlap across an entire layer plus prefetch depth. The host thread regains control immediately after enqueue (split-phase begin/end), so the CPU runs the shared expert and attention of the next layer concurrently with the GPU pipeline.

3.5 The PCIe scheduler

Two copy lanes share the link. Demand groups own a high-priority stream. Speculative uploads (below) ride a second stream and *yield between tiles* whenever a demand group is active: since hardware copy queues are FIFO and in-flight copies cannot be aborted [15], tile-granular yielding is the practical form of preemption, with interference bounded by Proposition 4.5. Stale speculative work is cancelled at the queue, the only place cancellation is possible.

3.6 Prefetch sources

Four signals keep the queue full, in priority order:

1. **Current router** — the demand path itself.
2. **Layer predictor** — applying layer $\ell+1$ ’s router to layer ℓ ’s post-attention state recalls 71.6% of the true top-8 (measured by colibrì [10]); a pilot thread loads predicted experts from disk and pushes them into the execution cache over the speculative lane *while it still exclusively owns the host buffer*, eliminating use-after-recycle races by construction.
3. **Temporal locality** — handled by retention rather than transfer: experts used for token n remain cached for token $n+1$ (§3.7), so the characteristic “447447” repetition of MoE routing becomes zero-traffic hits.

4. **Long-term heat** — persistent routing histograms place the globally hot set as *resident* tensors at startup (colibrì’s learned pin), a tier COLISTREAM deliberately preserves: the streaming cache serves the tail, not the head.

Because the execution cache sits *above* RAM, residency is probed at routing time: an expert present in VRAM but evicted from RAM skips its disk read entirely.

3.7 Replacement policy

Each slot carries a decayed frequency f and a last-use clock r . The victim minimizes the score

$$\sigma(\text{slot}) = (\min(2^{16}-1, \text{recency})) \ll 16 \mid \min(2^{16}-1, f), \quad f \leftarrow \lfloor f/2 \rfloor \text{ every } C \text{ evictions, } (1)$$

i.e. *recency-primary with frequency tiebreak and periodic decay*. This inverts the frequency-primary LFRU used by colibrì’s RAM hot store, deliberately: §4.3 proves that frequency-primary scoring self-destructs under working-set turnover, and §6 measures the failure at 7.6×. The division of labor is principled — long-term frequency belongs to the resident tier (source 4 above); the streaming cache’s dominant signal is the temporal locality of the demand stream itself.

3.8 Failure containment

Every stage degrades to the host path, never to a wrong answer: an oversized expert (e.g. the int8 multi-token-prediction head) is rejected per-expert via an acceptance mask; a group whose synchronization fails is recomputed on the CPU from host buffers whose lifetime provably spans the group; a failed speculative upload simply forgets the prediction; a stale residency probe is caught by a null-buffer guard and falls back to a fresh disk read. Output correctness never depends on the cache, the scheduler, or the predictor being right.

4 Analysis

4.1 Notation and model

A token demands M expert applications per MoE layer ($M \leq k \cdot \text{batch}$ after deduplication). Each expert has size s bytes. Demand misses cascade: a VRAM hit costs only kernel time c ; a RAM hit crosses PCIe at bandwidth B_p ; a full miss is first read from disk at B_d . Let h_V be the VRAM hit rate and h_R the conditional RAM hit rate given a VRAM miss. Per layer, $M_p = M(1 - h_V)$ experts cross PCIe and $M_d = M(1 - h_V)(1 - h_R)$ come from disk. We write τ for the service time of one tile at the slowest stage and m for the number of pipeline stages ($m=3$: disk, PCIe, compute).

Proposition 4.1 (Serial cost). *A load-then-execute engine that fetches and computes each expert synchronously spends, per layer,*

$$T_{\text{serial}} = M_d \frac{s}{B_d} + M_p \frac{s}{B_p} + M c. \quad (2)$$

Proof. Immediate: with no overlap, stage times add per expert, and experts add per layer. $\square$

Assumption 4.2. (i) The three stages run on independent engines (NVMe queue, DMA copy engine, SMs); (ii) each stage is work-conserving and FIFO within a demand group; (iii) at least two slots of buffering separate consecutive stages (double buffering); (iv) every transfer and read is decomposed into tiles of service time at most τ per stage.

Theorem 4.3 (Pipeline bound). *Under Assumption 4.2, the per-layer completion time of the streamed pipeline satisfies*

$$T_{\text{stream}} \leq \max\left(M_d \frac{s}{B_d}, M_p \frac{s}{B_p}, M c\right) + (m-1)\tau. \quad (3)$$

Proof. Index the N tiles of the layer’s demand set in FIFO order and let $\tau_j^{(i)}$ be tile i ’s service time at stage j , with stage busy times $\beta_j = \sum_i \tau_j^{(i)}$ (so $\beta_1 = M_d s / B_d$, $\beta_2 = M_p s / B_p$, $\beta_3 = M c$ up to per-tile rounding absorbed into τ). Let $F_j^{(i)}$ be tile i ’s finish time at stage j . With work-conservation and sufficient buffering, $F_j^{(i)} = \max(F_j^{(i-1)}, F_{j-1}^{(i)}) + \tau_j^{(i)}$. By induction on (i, j) , $F_j^{(i)} \leq \max_{j' \leq j} (\sum_{i' \leq i} \tau_{j'}^{(i')}) + \sum_{j' < j} \max_i \tau_{j'}^{(i)}$: the first term is bounded by the largest partial busy time, and each stage boundary contributes at most one maximal tile of fill delay. Evaluating at $i = N$, $j = m$ gives $T_{\text{stream}} = F_m^{(N)} \leq \max_j \beta_j + (m-1)\tau$. $\square$

Corollary 4.4 (Streaming speedup). *Ignoring the $O(m\tau)$ fill term, streaming converts the sum of stage times into their maximum: $T_{\text{serial}}/T_{\text{stream}} \rightarrow (\sum_j \beta_j) / \max_j \beta_j \in [1, m]$, maximized when stages are balanced. Conversely, when one stage dominates (e.g. $\beta_1 \gg \beta_2 + \beta_3$, a slow disk), no improvement to the other stages can exceed the Amdahl bound [1] $T_{\text{serial}}/\beta_1 = 1/\varphi$ where $\varphi = \beta_1/T_{\text{serial}}$.*

With the measured parameters of our evaluation platform ($B_d \approx 0.5$ GB/s, $B_p \approx 27$ GB/s pinned, $s = 18.9$ MB, $M = 600$ per token cold), $\beta_1 \approx 22.7$ s $\gg \beta_2 \approx 0.42$ s $\gg \beta_3$: cold decode is disk-bound by a factor of 50, and 10–20% CPU/GPU utilization is the *correct* signature of a saturated pipeline, not a defect. The tier’s leverage grows exactly as $h_V, h_R \rightarrow 1$ move β_1 off the critical path; the warm limit is $\max(\beta_2, \beta_3)$, measured in §6.1 at $\sim 16,000$ experts/s.

4.2 Tile size: overhead vs. preemption

Let c_t be the fixed cost of issuing one tile (API call, WDDM submission, copy-engine descriptor) and B_∞ the asymptotic link bandwidth. An expert of s bytes in tiles of t bytes transfers in time

$$T(t) = \frac{s}{B_\infty} + \left\lceil \frac{s}{t} \right\rceil c_t \quad \implies \quad B_{\text{eff}}(t) = \frac{B_\infty}{1 + c_t B_\infty / t}, \quad (4)$$

while a demand transfer arriving behind speculative traffic waits at most $W(t) = t/B_\infty + c_t$.

Proposition 4.5 (Feasible tile window). *For overhead fraction at most ε and preemption delay at most δ , the tile size must satisfy*

$$\frac{c_t B_\infty (1 - \varepsilon)}{\varepsilon} \leq t \leq B_\infty (\delta - c_t). \quad (5)$$

The window is nonempty iff $\delta \geq c_t/\varepsilon$ (up to the $(1-\varepsilon)$ factor); both bounds scale linearly in B_∞ , so faster links want proportionally larger tiles.

Proof. Directly from $B_{\text{eff}}(t) \geq (1 - \varepsilon)B_\infty$ and $W(t) \leq \delta$ in (4). $\square$

Fitting (4) by least squares to measured per-expert transfer times at $t \in \{0.5, 2, 4\}$ MB (Fig. 3) yields $B_\infty = 27.6$ GB/s and $c_t = 2.0 \mu\text{s}$ with $R^2 = 0.99$. At the default $t = 512$ KB this predicts 90% of asymptotic bandwidth with $W \leq 21 \mu\text{s}$ — three orders of magnitude below an expert’s transfer time, so speculative traffic is effectively invisible to demand latency.

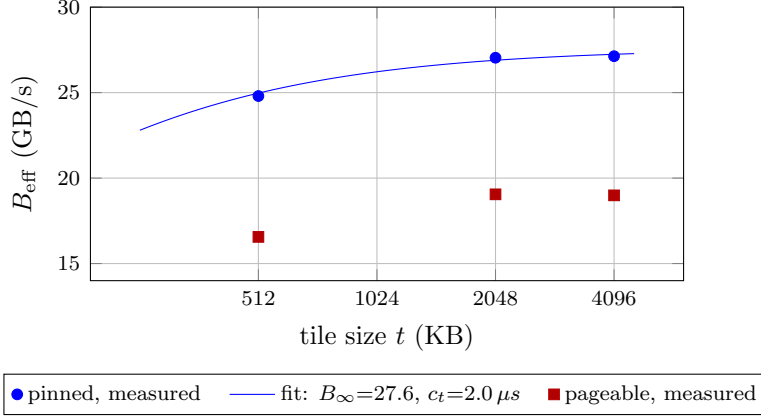


Figure 3: Effective streaming bandwidth vs. tile size on PCIe 5.0 x16 under Windows/WDDM (RTX 5070 Ti), with the two-parameter overhead model of Eq. (4). Page-locked host memory buys $\sim 1.45\times$ over pageable; the WDDM submission path, not the wire, sets B_∞ .

4.3 Frequency-primary replacement is unsafe under turnover

Proposition 4.6 (LFU pollution). *Consider a cache of C slots managed by any score in which frequency strictly dominates recency ($\text{victim} = \arg \min$ of $f \ll b \mid r$ with no decay), whose slots all hold items of frequency $f \geq 2$. Present a new working set of $C' \leq C$ distinct items, each requested repeatedly in round-robin passes. Then after the free slots are exhausted, every insertion evicts a frequency-1 member of the new working set, the new set never becomes resident, and its steady-state hit rate is 0 while the stale set retains the cache indefinitely.*

Proof. A new item enters with $f=1$. Any resident stale item scores at least $2 \ll b$, while any resident new item scores $1 \ll b+r < 2 \ll b$. The minimum is therefore always taken over new items; each insertion evicts one, so at most the (constant) number of initially free slots of the new set are ever simultaneously resident. Requests to the $C' - O(1)$ non-resident members miss forever; no stale item is ever the minimum, so none is ever evicted. $\square$

Remark 4.7. Periodic decay ($f \leftarrow \lfloor f/2 \rfloor$ every C evictions) bounds the pollution episode: after at most $\lceil \log_2 f_{\max} \rceil$ decay rounds every stale score falls to $1 \ll b$, ties break by recency, and the cache converges to LRU behavior over the live set. COLISTREAM adopts the stronger fix of recency-primary scoring (Eq. 1) — LRU over ties broken by decayed frequency — for which the new working set is resident after one pass, matching LRU-K/ARC-era wisdom that recency must be able to override stale frequency [16, 14]. We observed Proposition 4.6 in vivo before the fix: a full working-set turnover re-streamed 2.27 GB at 24.7 GB/s on every pass instead of hitting cache, a $7.6\times$ regression (§6.1).

4.4 Expected hit rate under Zipfian routing

Model expert selection as an independent reference stream over $N = L \cdot E$ (layer, expert) pairs with popularity $p_i \propto i^{-\alpha}$ (routing histograms of MoE models are heavy-tailed; colibri’s learned-pin gains attest to this). The Che approximation [4] for LRU gives a characteristic time T_C solving $\sum_i (1 - e^{-p_i T_C}) = C$ and hit rate

$$h(C) = \sum_i p_i (1 - e^{-p_i T_C}). \quad (6)$$

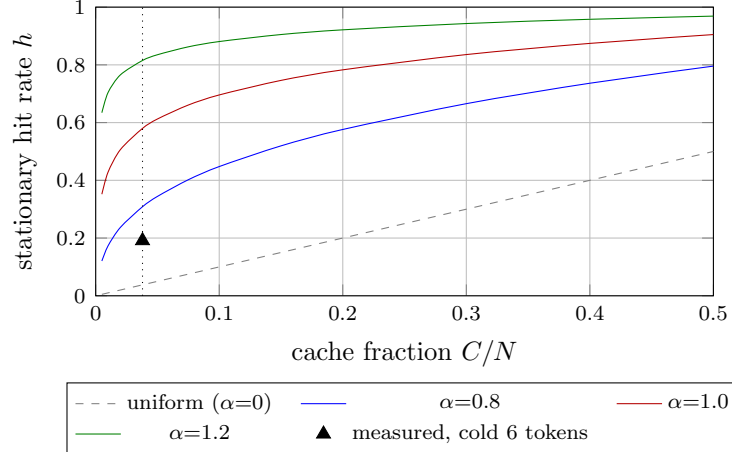


Figure 4: Che-approximation [4] LRU hit rate under Zipf(α) expert popularity, $N=19,200$. The dotted line marks our platform’s 727-slot pool ($C/N=0.038$); the triangle is the measured cold-start hit rate, a lower bound on the stationary value.

Figure 4 plots (6) for $N = 19,200$ (75 layers $\times$ 256 experts). At our platform’s $C/N = 3.8\%$, skew between $\alpha = 0.8$ and $\alpha = 1.2$ spans $h_V \in [0.31, 0.82]$ — the execution cache’s value is precisely the skew of routing. Our cold-start measurement (19.1% over the first six tokens, before any locality accumulates) sits below the stationary curves as expected; the oracle-model decode, whose working set fits entirely, reaches the $h_V \rightarrow 1$ regime at 88–90%.

5 Implementation

COLISTREAM is implemented inside colibri [10]: a ~ 600 -line CUDA module (slot pool, tile engine, dual copy lanes, policy, statistics) exporting nine C symbols, plus ~ 300 lines of engine integration (two-phase submit in the MoE block, the pilot-thread speculative hook, the RAM-above-VRAM residency probe, host-buffer page-locking with a budget). On Windows the module ships as a runtime DLL resolved symbol-by-symbol, so older binaries degrade gracefully; on Linux it links directly. All settings are command-line flags (`--stream`, `--stream-vram`, `--stream-tile-kb`, `--stream-pin-gb`) mirrored by environment variables. The engine’s existing tiers — learned-pin resident experts, RAM LRU, disk container — are preserved untouched; COLISTREAM slots in between RAM and compute.

Correctness methodology. The full stack is validated *token-exactly* against a reference `transformers` implementation of the same architecture: teacher-forcing 32/32 positions and greedy generation 20/20 tokens with the tier active, under async disk I/O and speculative prefetch. A standalone suite covers all four container formats, odd packed dimensions, eviction pressure, oversized-expert rejection, two concurrent demand groups, and a prefetch thread racing demand. As with any change of accumulation order, GPU output is not bit-identical to the CPU path; greedy argmax ties can therefore fork the token stream — every emitted token remains the argmax of a valid forward.

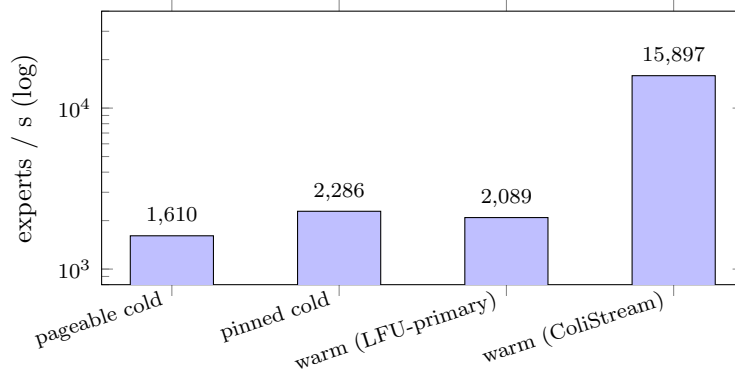


Figure 5: Standalone throughput, 192 GLM-shaped experts in groups of 8 (2 MB tiles). “Warm (LFU-primary)” is the pre-fix policy of Proposition 4.6: a full working-set turnover polluted the cache and every “warm” pass re-streamed at PCIe speed; the recency-primary policy serves it from VRAM, 7.6 $\times$ faster and with zero bus traffic.

6 Evaluation

Platform. RTX 5070 Ti (16 GB, PCIe 5.0 x16), AVX-512-VNNI CPU, 96 GB RAM, Windows 11, CUDA 13.1. Model: GLM-5.2 [21], 744B parameters, int4 container (370 GB, 75 MoE layers $\times$ 256 experts, $k=8$, 18.9 MB/expert), served from an NVMe whose measured cold random-read bandwidth is 0.49–0.51 GB/s *regardless of parallelism* (8/16/32 threads) — a deliberately adversarial, severely disk-bound configuration.

6.1 Microbenchmarks

Figure 5 summarizes the standalone pipeline on GLM-shaped synthetic experts (11.8 MB int4, groups of 8, demand path only). Cold streaming sustains 24.8–27.1 GB/s with all compute hidden behind the copies (Theorem 4.3’s max regime: kernels add no wall time); the warm execution cache serves the same groups kernel-bound at $\sim 16,000$ experts/s with zero PCIe traffic — a 7.6 $\times$ gap that is exactly the replacement-policy ablation of Proposition 4.6: under the frequency-primary score, the warm pass *re-streamed* every byte (91.9 ms per 192-expert turnover); under Eq. (1) it hits cache (12.1 ms).

6.2 End-to-end on the 744B model

We compare the engine’s optimized CPU expert path (AVX-512-VNNI kernels, async I/O pool) against COLISTREAM on identical settings: same prompt, greedy decoding, 16 tokens, speculation disabled on both sides, learned pin disabled, runs interleaved with controls to bound page-cache drift (CPU controls before and after the streamed run agree within 0.5%).

COLISTREAM is 1.065 $\times$ faster on a workload whose disk fraction is $\varphi = 264.2/277.3 = 0.953$. By Corollary 4.4, eliminating expert compute *entirely* could yield at most $1/\varphi = 1.049\times$; the additional margin comes from reduced disk service time (223 s vs. 264 s at identical 750 loads/token), i.e. deeper overlap of preads with the GPU pipeline once CPU cores stop competing with the I/O pool. Two honesty notes. First, Table 1’s top row shows how easily this comparison is inflated: engine defaults enable speculative decoding [13] only on the CPU side, and on a cold cache each verified draft routes extra experts (1216 vs. 750 loads/token), so a naive A/B credits

configuration	wall (s)	disk (s)	expert compute (s)	loads/token	tok/s
CPU path, default flags [†]	386.1	370.3	10.2	1216.5	0.041
CPU path, fair (DRAFT=0)	277.3	264.2	8.5	750.0	0.058
COLISTREAM	260.3	223.2	32.3 [‡]	750.0	0.061

Table 1: End-to-end A/B, cold-ish cache. [†]Confounded: without CUDA the engine defaults speculative decoding on, which on a cold cache inflates expert traffic $1.6\times$ — an A/B that ignores this reports a misleading $1.48\times$ advantage. [‡]Includes the GPU pipeline’s synchronization window, largely overlapped with disk service.

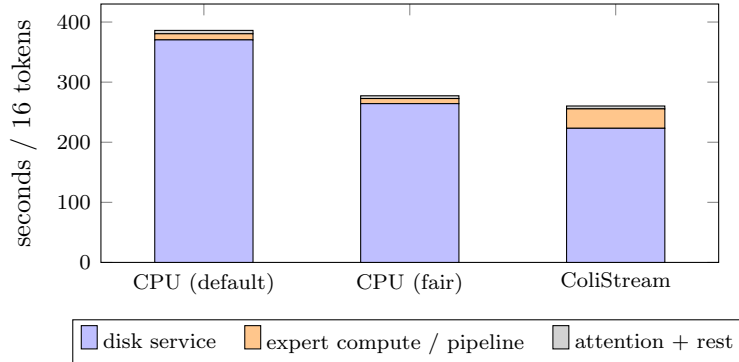


Figure 6: Wall-time decomposition of Table 1. The workload is disk-dominated ($\varphi = 0.95$); COLISTREAM’s win comes from removing expert compute from the critical path *and* from deeper disk/transfer overlap, together slightly exceeding the compute-only Amdahl ceiling.

COLISTREAM with $1.48\times$. Second, greedy paths can fork between kernel families (accumulation order), so per-run cache statistics are indicative, not paired.

The pilot prefetch (source 2) composes as designed: with cross-layer prediction active, speculative uploads land in the execution cache and are consumed by demand before eviction, at zero measured cost to demand latency (Proposition 4.5’s window).

6.3 Where the architecture pays

Corollary 4.4 predicts the regimes. On this platform the disk is $50\times$ slower than the link, so cold decode saturates the NVMe and *any* expert-compute change is bounded by $1.05\times$ — which COLISTREAM meets and slightly exceeds. On the same box with a page-cache-warm or RAM-resident expert set, the binding stage becomes $\max(\beta_2, \beta_3)$: the streamed pipeline’s 2,250 experts/s cold and $\sim 16,000$ experts/s warm against 600 expert-loads/token give ceilings of 3.7 and 26 tok/s respectively, versus a CPU expert path that our measurements place at ~ 8.5 s per 12,000 applications ($\sim 1,400$ experts/s) — a $1.6\text{--}11\times$ compute-side advantage that materializes exactly as the storage bottleneck is relieved (faster NVMe, larger RAM, higher h_V via Eq. (6)).

7 Limitations and Future Work

Single-GPU only; multi-device streaming with expert placement across links is open. In-flight copies cannot be cancelled — tile granularity bounds but does not eliminate speculative interference. The Windows/WDDM submission path caps B_∞ at 27.6 GB/s on a 64 GB/s link; a Linux/native

port of the measurement is future work. The execution cache stores whole experts; caching *tiles* would admit partial residency and adaptive precision (int1.5/int1 re-quantization for cold experts) at the cost of a two-level directory. Finally, our hit-rate analysis assumes an independent reference model; routing is in fact autocorrelated (which helps), and a Markov-modulated analysis would tighten §4.4.

8 Conclusion

COLISTREAM reframes MoE offloading: stop asking what fits in VRAM, and instead keep every engine of the memory hierarchy busy on a single stream of expert tiles. The mathematics is classical — pipeline bounds, cache approximations, Amdahl’s law — but their composition yields a design in which correctness never depends on prediction, interference is provably tile-bounded, and performance degrades to exactly the bottleneck the hardware imposes. On a 16 GB consumer GPU the architecture streams a 744B model’s experts at effective link speed, serves repeats at 16,000 experts/s from cache, and beats a heavily optimized CPU path even on a workload constructed so that almost nothing was winnable. The gap between that workload’s $1.065\times$ and the warm regime’s $11\times$ compute advantage is not a promise; it is a disk upgrade.

Acknowledgments. This work builds on the colibrì engine [10]; the resident tier, learned pinning, and router-lookahead measurement infrastructure are its author’s.

References

- [1] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *AFIPS Spring Joint Computer Conference*, 1967.
- [2] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, and Yuxiong He. DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2022. arXiv:2207.00032.
- [3] Laszlo A. Belady. A study of replacement algorithms for a virtual-storage computer. *IBM Systems Journal*, 5(2), 1966.
- [4] Hao Che, Ye Tung, and Zhijun Wang. Hierarchical web caching systems: Modeling, design and experimental results. *IEEE Journal on Selected Areas in Communications*, 20(7), 2002.
- [5] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [6] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. arXiv:2208.07339.
- [7] Artyom Eliseev and Denis Mazur. Fast inference of mixture-of-experts language models with offloading. *arXiv preprint arXiv:2312.17238*, 2023.
- [8] William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23, 2022. arXiv:2101.03961.

- [9] Elias Frantar, Saleh Ashkboos, Torsten Hoefer, and Dan Alistarh. GPTQ: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.
- [10] JustVugg. colibrì: Tiny engine, immense model — running a 744B MoE on consumer hardware. <https://github.com/JustVugg/colibri>, 2026.
- [11] Keisuke Kamahori, Yile Gu, Kan Zhu, and Baris Kasikci. Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models. *arXiv preprint arXiv:2402.07033*, 2024.
- [12] Dmitry Lepikhin, HyukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. GShard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.
- [13] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2211.17192.
- [14] Nimrod Megiddo and Dharmendra S. Modha. ARC: A self-tuning, low overhead replacement cache. In *USENIX Conference on File and Storage Technologies (FAST)*, 2003.
- [15] NVIDIA Corporation. CUDA C++ programming guide, v13.1. <https://docs.nvidia.com/cuda/>, 2025.
- [16] Elizabeth J. O’Neil, Patrick E. O’Neil, and Gerhard Weikum. The LRU-K page replacement algorithm for database disk buffering. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, 1993.
- [17] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. ZeRO-Offload: Democratizing billion-scale model training. In *USENIX Annual Technical Conference (ATC)*, 2021. arXiv:2101.06840.
- [18] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarsz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1701.06538.
- [19] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. FlexGen: High-throughput generative inference of large language models with a single GPU. In *International Conference on Machine Learning (ICML)*, 2023. arXiv:2303.06865.
- [20] Leyang Xue, Yao Fu, Zhan Lu, Luo Mai, and Mahesh Marina. MoE-Infinity: Offloading-efficient MoE model serving. *arXiv preprint arXiv:2401.14361*, 2024.
- [21] Z.ai. GLM-5.2: A 744B-parameter mixture-of-experts language model. Model release, <https://z.ai>, 2026.